Vaskaran Sarcar

Java Design Patterns

A Hands-On Experience with Real-World Examples

3rd ed.

Vaskaran Sarcar
Garia, Kolkata, India

ISBN 978-1-4842-7970-0 e-ISBN 978-1-4842-7971-7
https://doi.org/10.1007/978-1-4842-7971-7

This Apress imprint is published by the registered company APress Media, LLC, part of Springer Nature.

The registered company address is: 1 New York Plaza, New York, NY 10004, U.S.A.

First, I dedicate this book to Almighty GOD and the Gang of Four. Then I dedicate this work to all who have great potential to produce top-quality software but could not flourish for to various reasons. My message for them: "Dear reader, I want to hold your hands and help you express your hidden talents to the outside world."

Introduction

It is my absolute pleasure to write the third edition of *Java Design Patterns* for you. You can surely guess that I got this opportunity because you liked the previous edition of the book and shared your nice reviews from across the globe. So, once again I'm excited to join your design patterns journey. This time I present a further simplified, better organized, and content-rich edition to you.

You probably know that the concept of design patterns became extremely popular with the Gang of Four's famous book *Design Patterns: Elements of Reusable Object-Oriented Software* (Addison-Wesley, 1994). The book came out at the end of 1994, and it primarily focused on C++. But it is useful to know that these concepts still apply in today's programming world. Sun Microsystem released its first public implementation of Java 1.0 in 1995. So, in 1995, Java was new to the programming world. Since then, it has become rich with new features and is now a popular programming language. On the other hand, the concepts of design patterns are universal. So, when you exercise these fundamental concepts of design patterns with Java, you open new opportunities for yourself.

My end goal is simple: I want you to develop your programming skills to the next level using design patterns in your code. Unfortunately, this skill set cannot be acquired simply by reading. This is why I made this guide to the design patterns that you want to use in Java.

I have been writing books on design patterns since 2015 in different languages such as Java and C#. These books were further enhanced, and multiple editions of them were published and well received. In the initial version, my core intention was to implement each of the 23 Gang of Four (GoF) design patterns using simple examples. One thing was always in my mind when writing: I wanted to use the most basic constructs of Java so that the code would be compatible with both the upcoming version and the legacy version of Java. I have found this method helpful in the world of programming.

In the last few years, I have received many constructive suggestions from my readers. The second edition of this book was created with that feedback in mind. I also updated the formatting and corrected some typos

from the previous version of the book and added new content to this edition. In the second edition of the book, I focused on another important area. I call it the “doubt-clearing sessions.” I knew that if I could add some more information such as alternative ways to write these implementations, the pros and cons of these patterns, and when to choose one approach over another, readers would find this book even more helpful. So, in the second edition of the book, “Q&A Session” sections were added in each chapter to help you learn each pattern in more depth. I know you liked it very much.

So, what is new in the third edition? Well, the first thing I want to tell you is that since the second edition of the book is already big, this time I made the examples shorter and simpler. Also, I place the related chapters close to each other. This is why you’ll see the Chain of Responsibility pattern after the Observer pattern. The same is true for Simple Factory and Factory Method patterns, Strategy and State patterns, and Command and Memento patterns. In addition, at the beginning of the book, you’ll read a detailed discussion on SOLID design principles, which are used heavily across these patterns. Apart from these changes, I add more code explanations for your easy understanding.

Malcolm Gladwell in his book *Outliers* (Little, Brown and Company, 2008) talked about the 10,000-hour rule. This rule says that the key to achieve world-class expertise in any skill is, to a large extent, a matter of practicing the correct way for a total of around 10,000 hours. I acknowledge the fact that it is impossible to consider all experiences before you write a program. Sometimes, it is also ok to bend the rules if the return on investment (ROI) is nice. So, I remind you about the **Pareto** principle or the **80-20 rule**. This rule simply states that 80% of outcomes come from 20% of all causes. This is useful in programming, too. When you identify the most important and commonly used design patterns and use them in your applications properly, you can make top-quality programs. In this book, I discuss the programming patterns that can help you write better programs. You may know some of them already, but when you see them in action and go through the Q&A sessions, you’ll understand their importance.

How the Book Is Organized

The book has four major parts:

- Part 1 consists of the first two chapters, in which you will explore the SOLID principles and learn to use the Simple Factory pattern.
- Part 2 consists of the next 23 chapters, in which you learn and implement all of the Gang of Four design patterns.
- In the world of programming, there is no shortage of patterns, and each has its own significance. So, in addition to the SOLID principles and design patterns covered in Part 1 and Part 2, I discuss two additional design patterns (Null Object and MVC) in Part 3. They are equally important, commonly used, and well-known patterns in today's world of programming.
- Finally, in Part 4 of the book, I discuss the criticism of design patterns and give you an overview of anti-patterns, which are also important when you implement the concepts of design patterns in your applications. I also include a FAQ on design patterns.
- Starting from Chapter 2, each chapter is divided into six major parts: a definition (which is termed as "intent" in the GoF book), a core concept, a real-life example, a computer/coding world example, at least one sample program with various outputs, and the "Q&A Session" section. These "Q&A Session" sections can help you learn about each pattern in more depth.
- You can download the source code of the book from the publisher's website. I have a plan to maintain the errata and, if required, I can also make updates/announcements there. So, I suggest that you visit those pages to receive any important corrections or updates.

Prerequisite Knowledge

The target readers for this book are those who are familiar with the basic language constructs in Java and have an idea about the pure object-oriented concepts like polymorphism, inheritance, abstraction, encapsulation, and most importantly, how to compile or run a Java application in the Eclipse IDE. This book does not invest time in easily available topics, such as how to install Eclipse on your system, how to write a "Hello

World" program in Java, or how to use an `if-else` statement or a `while` loop. I mentioned that this book was written using the most basic features so that for most of the programs in this book, you do not need to be familiar with advanced topics in Java. These examples are simple and straightforward. I believe that these examples are written in such a way that even if you are familiar with another popular language such as C# or C++, you can still easily grasp the concepts in this book.

Who Is This Book For?

In short, you should read this book if the answer is "yes" to the following questions:

- Are you familiar with basic constructs in Java and object-oriented concepts like polymorphism, inheritance, abstraction, and encapsulation?
- Do you know how to set up your coding environment?
- Do you want to explore the design patterns in Java step by step?
- Do you want to explore the GoF design patterns? Are you further interested in learning about Simple Factory, Null Object, and MVC patterns?
- Do you want to examine how the core constructs of Java work behind these patterns?

Probably you shouldn't read this book if the answer is "yes" to any of the following questions:

- Are you absolutely new to Java?
- Are you looking for advanced concepts in Java excluding the topics mentioned previously?
- Do you dislike a book that has an emphasis on Q&A sessions?
- "I do not like the Windows operating system and Eclipse. I want to learn and use Java without them." Is this statement true for you?
- "I am already confident about GoF design patterns and other patterns that you mentioned earlier. I am searching for other patterns." Is this statement true for you?

Useful Software

These are the important software/tools I used for this book:

- I executed and started testing my programs using Java version 16.0.1 and the Eclipse IDE (version 2021-03 (4.19.0)) in a Windows 10 environment. When I started writing this book, they were the latest versions. It is a big book and when I finished the initial draft, Eclipse 2021-09 was the latest edition and I kept updating the software. Before I submitted the final version of the book, I tested the code in Java 17 (version 17.0.1). We can surely predict that version updates will come continuously, but these version details should not matter much to you because I have used the fundamental constructs of Java. So, I believe that this code should execute smoothly in the upcoming versions of Java/Eclipse as well.
- Anything that is the latest today will be old (or outdated) tomorrow. But the core constructs (or features) are evergreen. All new features are built on top of these universal features. So, I like to write code that is compatible with a wide range of versions using the basic language constructs. I understand that you may have a different thought, but I like this approach for various reasons. If you know the latest features, changing the code to them is easy. But the reverse is not necessarily true. Take another common example: when you provide support to your clients and fix code defects in an application, you cannot use the latest language constructs in almost every case, because the original product was created with a software version that is old now.
- You can download the Eclipse IDE from **www.eclipse.org/downloads/**. You'll see the page shown in Figure **FM-1**.

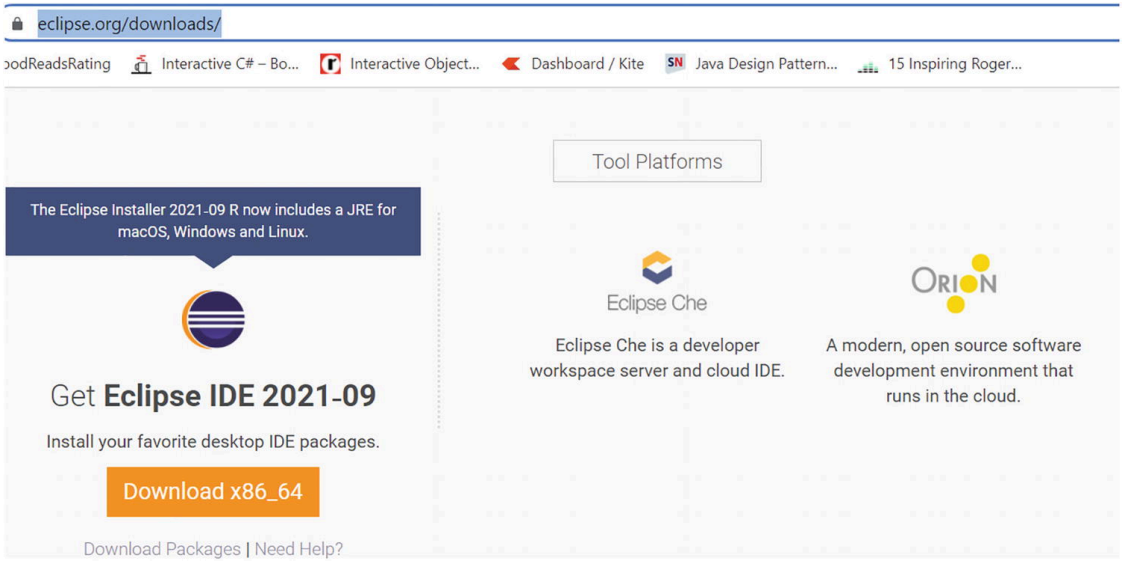


Figure FM-1 Download link for Eclipse

- Before I start coding, I use pen/pencils and paper. Sometimes, I use markers and a whiteboard. But when I show my programs in a book, I understand that I need to present these diagrams in a better shape. So, I use some tools to draw the class diagrams from my code. In the second edition of the book, I used ObjectAid Uml Explorer in the Eclipse editor. It is a lightweight tool for Eclipse. But it did not work for me with the updated versions of Eclipse. So, this time I used another nice tool, Papyrus. It is an open-source UML 2 tool based on Eclipse and licensed under the EPL. I was able to generate class diagrams easily using this tool. In some cases, to make them better, I added notes or edited a few things in the diagram so that you can understand it easily. For example, consider Figure **FM-2** (taken from Chapter **14**, when I discuss the Bridge pattern). You can understand easily that the Papyrus tool will not show you the markers for Hierarchy-1, Hierarchy-2, or BRIDGE inside the dashed rectangle. I edited the original diagram to help you understand the components better.

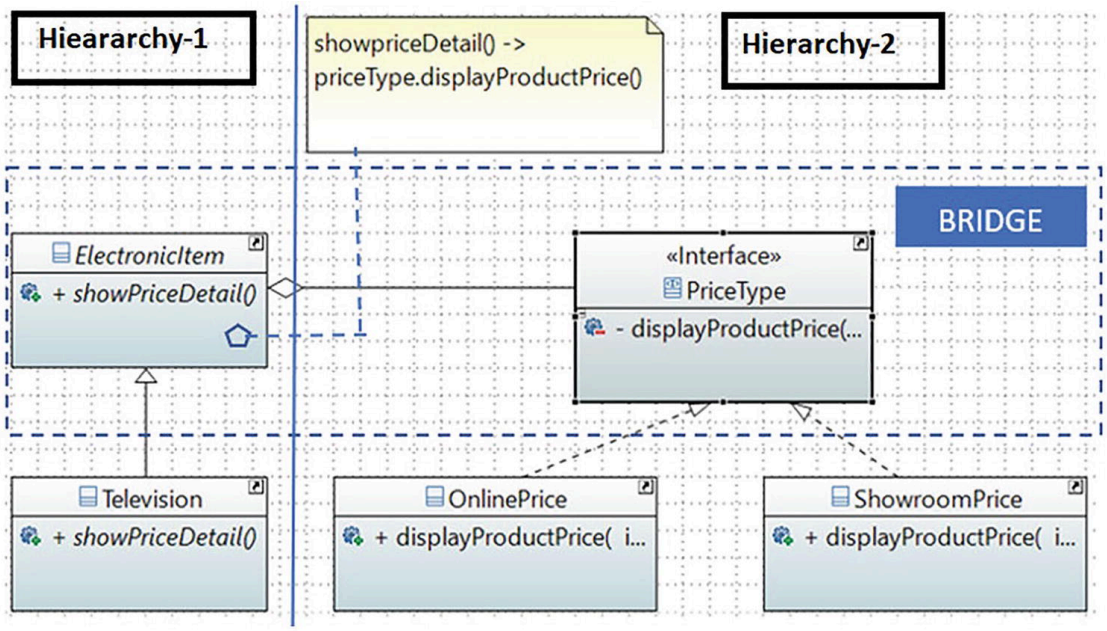


Figure FM-2 A class diagram that is taken from Chapter 14

- In short, these diagrams help you understand the code, but to learn design patterns, neither Papyrus nor Eclipse are mandatory. If you want to learn more about this reverse engineering process, you can refer to the following link: **https://wiki.eclipse.org/Java_reverse_engineering**.

> **Note** At the time of writing, all links in this book work and the information is correct. But these links and policies may change in the future.

Guidelines for Using This Book

Here are some suggestions so you can use the book more effectively:

- I assume that you have some idea about the GoF design patterns. If you are absolutely new to design patterns, I suggest you quickly go through Appendix A. This appendix will help you to become familiar with the basic concepts of design patterns.
- If you are confident with the coverage of Appendix A, you can start with any part of the book. But I suggest you go through the chapters sequentially. The reason is that some fundamental design techniques may be discussed in the Q&A Sessions of a previous chapter, and I do not repeat those techniques in later chapters.
- I believe that the output of the programs in this book should not vary in other environments, but you know the nature of software: it is

naughty. So, I recommend that if you want to see the exact same output, it's best if you can mimic the same environment.

Conventions Used in This Book

Here I mention only two points. In a very few places, to avoid more typing, I have used the word "he" only. Please treat it as "he" or "she", whichever applies to you.

To execute a program, I put all parts in the same folder/package. So, in most cases, I chose the package-private visibility. But if you want, you can increase the respective visibilities to public to reuse those parts. I used separate packages for separate programs to help you find all parts of a program at the same place.

Finally, all the output and code of the book follow the same font and structure. To draw your attention, in some places, I have made them bold. For example, consider the following output fragment (taken from Chapter **15**, when I discuss the Template Method pattern) and the line in bold:

Template Method Pattern with a hook method.

Computer Science course structure:

1. Mathematics
2. Soft skills
3. Object-Oriented Programming
4. **Compiler construction.**

Electronics course structure:

1. Mathematics
2. Soft skills
3. Digital Logic and Circuit Theory

Final Words

I must say that you are an intelligent person. You have chosen a subject that can assist you throughout your career. If you are a developer/programmer, you need these concepts. If you are an architect of a software organization, you need these concepts. If you are a college student, you need these concepts, not only to score high on exams but to enter the corporate world. Even if you are a tester who needs to take care of white-box testing or simply needs to know about the code paths of a product, these concepts will help you a lot.

This book is designed for you in such a way that upon its completion, you will have developed an adequate knowledge of the topic, and most importantly, you'll know how to proceed further. Remember that this is just the beginning. As you learn about these concepts, I suggest you write your own code; only then will you master this area. There is no shortcut for this. Do you remember Euclid's reply to the ruler? If not, let me remind you of his reply: **There is no royal road to geometry.** So, study and code. Understand a new concept and code again. Do not give up when you face challenges. These are the indicators that you are growing better.

Lastly, I hope that this book can help you and you will value the effort.

Any source code or other supplementary material referenced by the author in this book is available to readers on GitHub via the book's product page, located at www.apress.com/978-1-4842-7970-0. For more detailed information, please visit www.apress.com/source-code.

Acknowledgments

At first, I thank the Almighty. I sincerely believe that with HIS blessings only, I could complete this book. I extend my deepest gratitude and thanks to the following people.

Ratanlal Sarkar and Manikuntala Sarkar: My dear parents, with your blessings only, I could complete the work.

Indrani, my wife; **Ambika**, my daughter; **Aryaman**, my son: Sweethearts, I love you all.

Sambaran, my brother: Thank you for your constant encouragement.

Sekhar, Harsha, Abhimanyu, Carsten: As technical advisors, whenever I was in need, your support was there. Thank you one more time.

Sunil Sati, Anupam, Ritesh, Ankit: Sunil is ex-colleague cum senior who wrote the foreword for the second edition of this book. The others are my friends and technical advisors. Although this time you were not involved directly, still I acknowledge your support and help in the development of *Java Design Patterns* first edition and second edition.

Celestin, Laura, Aditee: Thanks for giving me another opportunity to work with you and Apress.

Sherly, Vinoth, Siva Chandran: Thank you for your exceptional support to beautify my work. Thank you all. Your efforts are extraordinary.

Table of Contents

About the Author

Vaskaran Sarcar

obtained his Master of Engineering in software engineering from Jadavpur University, Kolkata (India) and an MCA from Vidyasagar University, Midnapore (India). He was a National Gate Scholar from 2007-2009 and has more than 12 years of experience in education and the IT industry. Vaskaran devoted his early years (2005-2007) to the teaching profession at various engineering colleges. Later he joined HP India PPS R&D Hub Bangalore. He worked there until August 2019. At the time of his retirement from HP, he was a Senior Software Engineer and Team Lead. To follow his dream and passion, Vaskaran is now an independent full-time author. Other Apress books by him include

- *Simple and Efficient Programming in C# (Apress, 2021)*
- *Design Patterns in C# Second Edition (Apress, 2020)*

- *Getting Started with Advanced C# (Apress, 2020)*
- *Interactive Object-Oriented Programming in Java Second Edition (Apress, 2019)*
- *Java Design Patterns Second Edition (Apress, 2019)*
- *Design Patterns in C# (Apress, 2018)*
- *Interactive C# (Apress, 2017)*
- *Interactive Object-Oriented Programming in Java(Apress, 2016)*
- *Java Design Patterns (Apress, 2016)*

The following list is of his non-Apress books:

- *Python Bookcamp (Amazon, 2021)*
- *Operating System: Computer Science Interview Series (Createspace, 2014)*

About the Technical Reviewers

Abhimanyu

is a self-

motivated technological enthusiast with over 13 years of experience in software development. He is an expert in building big data solutions and large-scale machine learning applications, especially in the retail domain. Currently, Abhimanyu is building data-aware apps and solutions for the world's largest retailer.

Along with his work, he also likes astrophotography, speed-cubing, and playing his acoustic guitar in his free time.

Carsten Thomsen

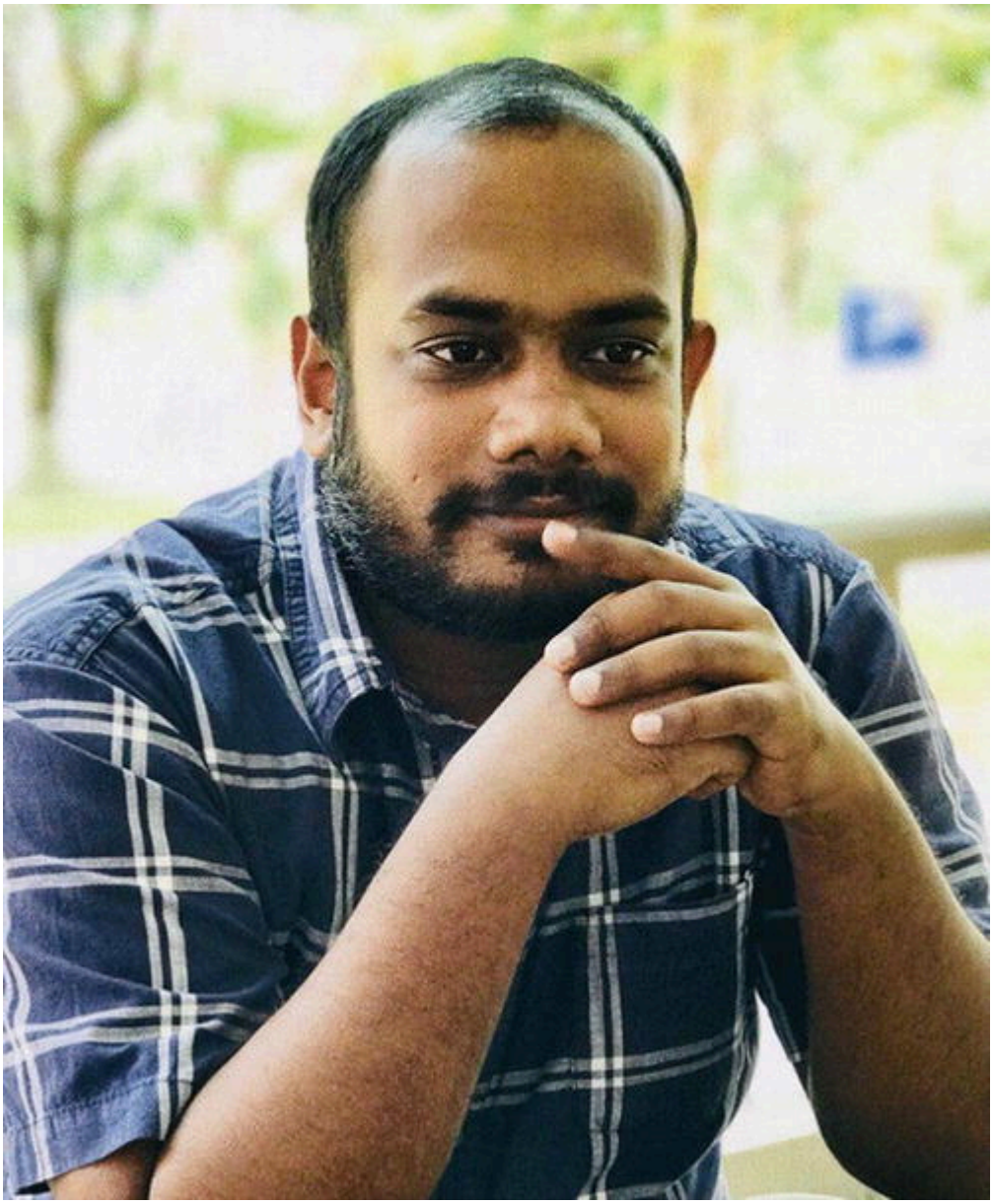
is a back-end developer primarily working with smaller front-end bits as well. He has authored and reviewed a number of books and created numerous Microsoft Learning courses, all to do with software development. He works as freelancer/contractor in various countries in Europe, using Azure, Visual Studio, Azure DevOps, and GitHub. Being an exceptional troubleshooter by asking the right questions, including the less logical ones, in a most-logical-to-least-logical fashion, he also enjoys working with architecture, research, analysis, development, testing, and bug fixing. Carsten is a very

good

communicator with great mentoring and team-lead skills, and great skills researching and presenting new material.

Harsha Jayamanna

has more than seven years of software engineering experience. Java, Spring, JakartaEE, Microservices, and Cloud are several of his expertise areas. He started his career in Sri Lanka and then, after several years, moved to Singapore. Currently, he is working as a software consultant in Sydney, Australia. Writing blog articles, reading technical books, and learning new technologies are his other areas of interest.He can be reached via `https://www.harshajayamanna.com/`.



Shekhar Kumar Maravi

is a Lead Engineer in Design and Development whose main interests are programming languages, algorithms, and data structures. He obtained his Master's degree in Computer Science and Engineering from Indian Institute of Technology Bombay. After graduation, he joined Hewlett-Packard's R&D Hub in India to work on printer firmware. Currently he is a technical lead engineer for automated pathology lab diagnostic devices at Siemens Healthcare R&D division. He can be reached by email at `shekhar.maravi@gmail.com` or via LinkedIn at **`www.linkedin.com/in/shekharmaravi`**.

Previous chapter

‹ Cover

Next chapter

Part I. Foundation ›